Hello, everyone. My name is Yevgen Prudyus**. I'm going to walk you through a three-tier web application, built entirely with AWS Native Services**. This presentation is especially for those who might be new to AWS. Our goal is to understand not just the components, but how this structure delivers critical capabilities essential for modern, high-performance applications So, to begin, let's ensure we're all on the same page about what a **three-tier architecture** is**.**

Go to Slide 2

It's a well-established pattern, fundamental to many user-facing applications. It consists of three distinct, interconnected tiers: web tier, application tier, and database tier.

1. **The Web Tier (or Presentation Tier):** This is the component that users directly interact with. It's the face of your application. In our demonstration, this will be a dynamic web page, but it could be anything from a mobile app's user interface to a desktop application.
2. **The Application Tier (or Logic Tier):** The 'brains' of the operation. It's the engine room where the actual processing happens. The code here translates user actions from the UI into functional operations. For instance, when a user interacts with our web page UI and makes a request (like submitting a form or requesting data), the application tier will handle that action, and then communicate with the database tier**. In our demo**, this tier will handle simple CRUD (Create, Read, Update, Delete) database operations and some light data processing.
3. **The Database Tier (or Data Tier):** This is where your application's data is stored, managed, and retrieved. It's the persistent storage layer. **For our application**, it will hold simple transactional data relevant to the application's function.

Go to Slide 3

The beauty of this layered approach is that it allows us to achieve four key characteristics: **scalability** (growing to meet demand), **high availability** (always accessible), **resiliency** (recovering from failures), and **security** (protecting our data and services). *I'll be touching on how AWS helps us achieve these as we walk through the demo in the AWS console***.**

Go to Slide 4
[Slide: Overall Architecture Diagram]

(Show Architecture Diagram) *This diagram presents the comprehensive view of the architecture we will explore.* Let's trace the flow of traffic to understand it. A user first interacts with our application through the **external-facing Application Load Balancer's DNS entry**. This load balancer is our front door. It forwards client traffic to our **Web Tier EC2 instances**. These instances are running NGINX web servers, configured to serve a React.js website. Importantly, our web tier also redirects API calls from the client to the **Application Tier's internal-facing Application Load Balancer**. This internal load balancer then forwards that traffic to the **App Tier EC2 instances**. The app tier, written in Node.js, processes these requests, manipulates data, and retrieves or stores data in our **Amazon Aurora MySQL databases**. The result is then returned back through the tiers to the user interface. You'll also see components like load balancing, Auto Scaling Groups, which I'll explain later.

But before diving into the tiers themselves, the absolute foundation, the skeleton of this entire architecture, is the **networking** piece. This includes our VPC (**Amazon Virtual Private Cloud (VPC)**) and our subnets.
I'm going to now take you through these pieces in the AWS console to see what our networking looks like set up.

Go to AWS console
So this is the AWS console

(Transition to AWS Console - VPC Dashboard) I've built everything in the US East region, which is `us-east-1`. Typically, you'd choose the AWS Region geographically closest to your end-users to minimize latency. (show region)

Go to VPC

Let's navigate to the VPC console. You'll see we have one VPC created for this demo. An Amazon VPC lets you provision a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. This virtual network closely resembles a traditional network that you might operate in your own on-premises data center, but with the benefits of using AWS's scalable infrastructure. To create this VPC, we simply specified an IP address using **CIDR** range – for this demo, `10.0.0.0/16`. This range needed to be large enough be able to allow for enough IPs for our applications, and importantly, to be able to break up this CIDR range into six different subnets.

Go to Subnets

[AWS Console: Subnets]

Now, looking at our subnets, you'll see six of them. You might wonder, "Why six subnets for a three-tier architecture?" .When designing any application, you must design for failure to build a highly available and resilient system. In AWS, we achieve this by deploying our architecture across multiple Availability Zones. An AZ consists of one or more discrete data centers, each with redundant power, networking, and connectivity, housed in separate facilities within an AWS Region. Using multiple AZs allows AWS customers to operate production applications and databases that are more highly available, fault-tolerant, and scalable than would be possible from a single data center. It's a fundamental best practice. For this demo, I've used `us-east-1a` and `us-east-1b`. So, for each of our three logical tiers, we have two subnets, one in each AZ:

- Two **public subnets** for our web tier instances.
- Two **private subnets** for our application tier instances.
- Two **private subnets** for our database tier instances.

The difference between "public" and "private" subnets is critical for security and accessibility. Public subnets are configured to have direct access to the internet, while private subnets are not directly reachable from the internet.

Simply having a VPC and subnets isn't enough; we need to direct traffic. Our web instances in public subnets need internet access. This introduces a few more components: **Internet Gateways, NAT Gateways, and Route Tables.**

Go to Internet Gateway
[AWS Console: Internet Gateways]

Let's start with the **Internet Gateway (IGW)**. An IGW is a horizontally scaled, redundant, and highly available VPC component that allows communication between resources in your VPC and the internet. You can see I have one IGW and attached it to the VPC that we created.

[AWS Console: Route Tables]

Now in order to route traffic from your subnets to the internet , each subnet must be associated with a route table that includes a route for all traffic to the IGW.

For our public subnets (where our web tier lives) to actually use the IGW, their associated **Route Table** must contain a route to the IGW. To be clear, route table is a set of rules, called routes, that are used to determine where network traffic is directed. We have three main route tables defined.

Let's look at the `public-route-table. T`his route means is that all traffic destined for outside of our VPC is directly connected to the internet gateway. To be directed towards our internet gateway. Any subnet associated with this route table is considered a public subnet. If we check the subnet associations for this route table, we see both our `public-subnet-az1` and `public-subnet-az2` are associated.

[AWS Console: Private Route Table Details]

Now, for our private subnets (app and database tiers). For security, we don't want direct internet access to these. That's just not a secure way to build your application. However, our a private instances might need internet access for things like software updates.
To enable this, we use **NAT (Network Address Translation) Gateways**.

[AWS Console: NAT Gateways]

A NAT Gateway allows instances in a private subnet to connect to the internet but prevents the internet from initiating a connection with those instances. *Remember that when you launch a NAT Gateway, you must place them in your public subnets and not your private subnets. They are literally a gateway between your public and private subnets, so mistakenly placing them in a private subnet will not provide you internet connectivity. Also a single NAT service can only run within a single subnet.* We've created two NAT Gateways, one in each public subnet in each AZ for redundancy. *if one public subnet goes down, other private subnets would still have internet connectivity through their respective public subnets* then we created two route tables for each of the private app instance subnets in each of the availability zones. So that's why, it says route all traffic destined for outside of our VPC to that NAT gateway in that availability zone. And if you look at our subnet associations here, we've associated that private subnet of that availability zone with this

route table The same setup exists for AZ2. So that's kind of the skeleton in which we're going to build the rest of our architecture.

[Diagram Transition: Focus on Web Tier]

Now that our network and routing configurations are complete. Let's return to our diagram,

Go to Slide 5

let's now focus on the first tier: the **Web Tier**. this web tier contains a handful of components here, EC2 instances, load balancer, and auto scaling group. So let's take that step by step to see how this was built out and what exactly we achieved using these components. Let's navigate to the EC2 dashboard.

Go to EC2 DashBord

[AWS Console: Load Balancers]

(Navigate to EC2 Dashboard - Load Balancers) Following the flow of traffic, the entry point to our application is the **external-facing Application Load Balancer (ALB) DNS entry**. for those of you that may not be familiar , Elastic Load Balancing automatically distributes incoming application traffic across multiple targets, such as our EC2 instances, in multiple Availability Zones. It serves two main purposes here: managing traffic volume and performing continuous health checks

[AWS Console: Load Balancer Listeners]

Our external ALB is listening for HTTP traffic on port 80. When it receives a request, it forwards that traffic to a **Target Group**.

[AWS Console: Target Groups]

And if we click on the target group, This target group, `web-tg`, consists of our web tier EC2 instances. And we click on it again, and we see our two web instances, one in `us-east-1a` and one in `us-east-1b` (one in each Availability Zone), both marked as "healthy." The ALB performs **health checks** on these instances. If an instance becomes unhealthy, the load balancer automatically stops sending traffic to it and reroutes to the remaining healthy instances. This is key for **resiliency**.

Of course, we want to ensure we always have two healthy instances available. This is where Auto Scaling Groups provide critical value. let me showing you our auto scaling group.

Go to Auto Scaling Groups

[AWS Console: Auto Scaling Groups]

(Navigate to EC2 Dashboard - Auto Scaling Groups)

(Let's look at our Web Tier Auto Scaling Group. The core configuration specifies our scaling policy: a minimum of two instances, a desired capacity of two, and a maximum of two. This policy ensures we always maintain two healthy web instances across our defined Availability Zones.)

We have an ASG for our web tier, `web-asg`. The key configurations here are:

- **Desired Capacity:** The number of instances the ASG will try to maintain. We have it set to 2.
- **Minimum Capacity:** Set to 2.
- **Maximum Capacity:** Also set to 2 for this demo (but could be higher for production scaling).
- **Availability Zones:** Configured to launch instances across `us-east-2a` and `us-east-2b`.

If an instance were to terminate, the ASG would detect this and automatically launch a new instance to replace it. How does the ASG know *what kind* of instance to launch? This is defined by a **Launch Configuration** (or Launch Template).

[AWS Console: Launch Configuration Details]

If we go into here, these are the details of our launch configuration,we see this AMI ID

Our `web-launch-config` specifies:

- The **AMI (Amazon Machine Image) ID:** An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. We created this AMI by first launching a standard Amazon Linux 2 instance, installing NGINX, deploying our React code, configuring it all, and then creating an image from that instance. And then we take that image, and we created a launch configuration.

  The Launch Configuration also specifies the instance type and required permissions. This standardization is the key to quick and consistent deployment.

(If time permits, I can terminate a web instance to show the ASG automatically launching a replacement. You'd see it terminate, and shortly after, a new one would start initializing.)

[Demo Action: Terminate a Web Instance]

To demonstrate the system's self-healing capability, *I can terminate a web instance to show the ASG automatically launching a replacement. You'd see it terminate, and shortly after, a new one would start initializing* I will now terminate one of the running web instances.

[AWS Console: EC2 Instances (showing terminating instance)]

And I'm going to delete one web instance. So, that's my web instance. Go ahead and terminate that. Now that takes a few minutes, it already disappeared from running. So it's shutting down. Auto Scaling will detect that the desired capacity is no longer met and will automatically provision and launch a new instance based on the Launch Configuration.

[Demo Transition: Discuss App Tier while new instance launches]

While the new web instance is coming online, let's discuss the Application Tier.

Go to Slide 6

[Diagram Transition: Focus on App Tier]

Application Tier is very similar to the web tier, But the differences are in the details. Traffic from our web instances is directed to an **internal-facing Application Load Balancer**.
let's go to Load Balancer

Go to Load Balancer
[AWS Console: Load Balancers (showing internal ALB)]

"Internal" means it only has a private IP address and can only be accessed from within our VPC. This is a security best practice. This internal ALB listens for traffic (e.g., on port 80 as configured by the web tier) and forwards it to its target group, `app-tg`. The instances in this target group are configured to listen on the port our Node.js application uses (e.g., port 4000).
,

[AWS Console: App Target Group Targets]

Like the web tier, our app tier EC2 instances are managed by their own Auto Scaling Group (`app-asg`) using its own Launch Configuration (`app-launch-config`) and a different AMI, which contains the Node.js application code and its dependencies. These instances reside in the **private app subnets** and therefore do not have public IP addresses.

[Demo Action: Connect to App Instance via Session Manager and ping Google]

So, how do we access instances that don't have public IPs, or even those that do, without opening SSH ports? For this demo, and as a best practice, we use **AWS Systems Manager Session Manager**. Session Manager is a fully managed AWS capability that lets you manage your EC2 instances through an interactive one-click browser-based shell or through the AWS CLI. The key benefits are:

- **Enhanced Security:** No need to open inbound SSH ports or RDP ports.
- **No Bastion Hosts:** You don't need to set up and maintain bastion hosts (jump servers).
- **No SSH Key Management:** No need to manage SSH keys.
- **Centralized Access Control:** Access is controlled via IAM policies.

[AWS Console: Session Manager Shell]

Connecting to an App instance... Now, let's perform a simple network test: ping Google.

to demonstrate that the instance can reach the internet via the NAT Gateway, even though it has no public IP. I'm just going to hit connect here. It'll take me straight to that shell that I was talking about. And I'm just going to show you that it does have internet access. I'm going to ping Google and we see that packets are moving. So that's just a nice demonstration that we can in fact access the internet. . However, thanks to the NAT Gateway and its route table configuration, it can indeed initiate outbound connections.

[Demo Transition: Return to EC2 Instances to check on terminated Web Instance]

Now let's come back to our EC2 instances so I can show you. See here, this one doesn't have a name, this instance here, and it's initializing. So this is our web instance. I killed the web instance just to remind you, the autoscaling took care of bringing another one up. And you can see here that I terminated one in US East 1A and it brought exactly the same thing back up in US East 1A. So this is a really nice resiliency piece we have here, the self-healing nature of this application

[Diagram Transition: Focus on Database Tier]

Finally, let's talk about our **Database Tier**. **(Navigate to RDS Console)** For our database, we are using **Amazon Aurora with MySQL compatibility**, which is a part of Amazon RDS (Relational Database Service). RDS is a web service that makes it easier to set up, operate, and scale a relational database in the AWS Cloud.

[AWS Console: RDS Subnet Groups]

In order to create our Aurora database, we first created a **DB Subnet Group**. This tells RDS which subnets (and therefore AZs) within your VPC it can use for the database. We restricted this to our two private database subnets, `private-db-subnet-az1` and `private-db-subnet-az2`. Our Aurora database cluster, `demo-db-cluster`, has two instances:

1. A **Writer Instance:** writer nodes endpoint will allow you to read and write from that database
2. A **Reader Instance (Read Replica):** That is just a reader node specifically designed to help you distribute heavy read traffic.

A key feature of Aurora for **high availability and resiliency** is automatic failover. If our writer instance becomes unavailable (e.g., due to an AZ outage or instance failure), Aurora can automatically promote the reader instance in the other AZ to become the new writer. Your application would then need to be configured to connect to the new writer and just swap

out the endpoint that you're using .This ensures minimal downtime for your database.

[Diagram Transition: Focus on Security (perhaps highlighting security groups)]

The final pillar I want to cover is **Security Groups**. **(Navigate to EC2 - Security Groups, or VPC - Security Groups)** Security Groups act as virtual stateful firewalls for your instances to control inbound and outbound traffic at the instance level (and for other resources like ALBs and RDS). "Stateful" stateful, that means that anything that you allow to flow out of your instance is allowed by default to flow back in your instance.

[AWS Console: Security Groups Filtered by VPC]

Let's look at the Security Groups associated with our VPC.

[AWS Console: External Load Balancer Security Group Inbound Rules]

Let's go starting with our external load balancer, the entry point for our application, this is my IP right now. And it's allowing me to access port 80. that is the port that our load balancer is listening on. So just to demonstrate that I have access to this app, it's great, and it's working.

[Demo Action: Modify Security Group to remove IP access]

But, if I edit the security group and remove the rule allowing access from my IP, and I take that

DNS entry, and I open up a new browser window and try to access it, you'll see that I no longer have access to my application. So for the purpose of this demo, that was a really simple way to restrict access to my application. And so that is how we controlled the security. And I just showed you the security group for the load balancer, but what happens after that?

[AWS Console: Web Tier Security Group Inbound Rules]

After the load balancer, our traffic goes and gets forwarded to our web tier instances. And so the security group for, our public web tier instances should only allow traffic from our external load balancer security group. And then we just follow that pattern. The next layer only allows traffic from the previous layer security group. And then the layer after that, our database, for example, will only allow access from our private instances. So let's see, here's our database instance security group as an example. And we see that it's only allowing access from our private instance security group. Great.

We follow a principle of least privilege, only allowing necessary traffic:

1. **External ALB's Security Group (`external-alb-sg`):**
    - Inbound: Allows HTTP (port 80) from `0.0.0.0/0` (any IPv4 address) so users can access our website. For a demo, you might restrict this to your IP.
    - Outbound: Allows all traffic (default).
2. **Web Tier EC2 Instances' Security Group (`web-ec2-sg`):**
    - Inbound: Allows HTTP (port 80) *only* from the `external-alb-sg`. This ensures only the load balancer can send traffic to the web servers.
    - Outbound: Allows traffic to the `internal-alb-sg` on its listening port.

3. **Internal ALB's Security Group (`internal-alb-sg`):**
    - Inbound: Allows traffic on its listening port (e.g., 80 or 4000) *only* from the `web-ec2-sg`.
4. **App Tier EC2 Instances' Security Group (`app-ec2-sg`):**
    - Inbound: Allows traffic on the application port (e.g., 4000) *only* from the `internal-alb-sg`.
    - Outbound: Allows traffic to the `db-sg` on the database port.
5. **Aurora Database's Security Group (`db-sg`):**
    - Inbound: Allows MySQL traffic (port 3306) *only* from the `app-ec2-sg`. This ensures only our application tier can connect to the database.

[Diagram Transition: Overall Architecture Diagram]

To recap, we've explored a foundational 3-tier architecture- web application on AWS, demonstrating how the strategic use of services like VPC, EC2, Load Balancing, Auto Scaling, RDS Aurora, and Security Groups delivers the essential pillars of Scalability, High Availability, Resiliency, and Security.

This architecture provides a solid foundation. For production readiness, this can be extended with additional layers – perhaps incorporating a Content Delivery Network for global performance optimization, enhancing the security posture with Web Application Firewalls or API Gateways, or integrating managed services for user authentication and domain management.

Thank you for your time. I am available to answer any questions you might have.

[Diagram Transition: Overall Architecture Diagram]